TheoInf 1
Woche 13 (letzte Vorlesungswoche)

Korrekturen zu

Allgemeine Grammatiken

Grammatik für $\{a^n b^n c^n \mid n \in \mathbb{N}_0\}$

▸ Vorschlag letzte Vorlesung:

S → aSXY | ε
YX → XCX
CX → C ⇦ da liegt das Problem
aX → ab
bX → bb
Y → c
C → c

Beispiel:
S
aSXY
aaSXYXY
aaaSXYXYXY
aaaXYXYXY
aaaXXCXYXY
aaaXXCXXCXY
aaaXXCXCXY
aaaXXCCXY
aaaXXCCY
…
aaabbccc 😳

▸ neuer Vorschlag

S → aSBC | ε
CB → CX
CX → BX
BX → BC
} vertausche CB zu BC
aB → ab
bB → bb
C → c

Fortsetzung

Reterna

▸ Reterna ist Turing-mächtig

… kann also abstürzen

```
f(x) = f(x)
```

```
f(x) = x? f(x+1) : 0
```

"Stack Overflow"

"Call Stack", auf dem die Parameter und die "Rücksprungadressen" gespeichert werden

▸ Reterna ⇒ While

Reterna-Programm in While simulieren. Dazu benötige ich in While einen Stack (den Call Stack), kann ich z.B. mit einer Liste implementieren.

▸ While ⇒ Reterna

Gegen ein While-Programm.
Seien $x_1, x_2, \ldots, x_n$ die Variablen, die in diesem While-Programm verwendet werden.
Eine Teilmenge $\{y_1, y_2, \ldots, y_m\} \subseteq \{x_1, \ldots, x_n\}$ davon ist Eingabe; $m \leq n$
Alle anderen Variablen werden im Programm irgendwo initialisiert.
Eine der Variablen – x_k – ist die Variable out, die immer, wenn das Programm hält, die Ausgabe des Programms beinhaltet.

Ziel: konstruiere eine Reterna-Funktion f_k, die dasselbe berechnet wie das While-Programm.

▸ 1. Wandel While-Programm so um, dass kein *stop* (d.h. auch kein *return*) darin vorkommt.

Beispiel:
L = {w ∈ Σ* | w enthält nicht den Buchstaben A}

dazu While-Programm (mit *stop*):

```
i ← 0
while i < len(in)
   if in[i] = "A"
      return false
   end
   ++i
end
return true
```

kann man in folgendes While-Programm ohne *stop* umwandeln:

```
i ← 0
b ← true
while b and (i < len(in))
   if in[i] = "A"
      b ← false
   end
   ++i
end
```

Am Ende soll b als Rückgabewert dienen.

▶ **2.** alle Variablen zu Parametern machen
Deligiere die Funktion mit m Parameter an eine mit n Parametern, z.B.:

$f(y_1, \ldots, y_m) = f_k(\underbrace{0, \ldots, 0, y_1, 0, \ldots, 0, y_m, 0, \ldots, 0}_{x_1, \ldots, x_n})$

⇦ hier werden alle Variablen x_i, die nicht in der Menge $\{y_1, \ldots, y_m\}$ vorkommen, auf 0 gesetzt.
⇦ da kann ich natürlich in der Praxis schon dafür verwenden, diese Variablen auf die richtigen Werte zu initialisieren.

Beispiel: (von oben)
$f(in) = f_3(in, i, b)$

$in = x_1$
$i = x_2$
$b = x_3 = ret$

▶ **3.** Bei einfachen While-Programmen ohne *if* und *while* ist es leicht, eine Reterna-Funktion zu definieren

Beispiel:

```
++x2
--x3
x4 ← x1 + x2
```

$f_1(x_1, x_2, x_3, x_4) = x_1$
$f_2(x_1, x_2, x_3, x_4) = x_2 + 1$
$f_3(x_1, x_2, x_3, x_4) = x_3 - 1$
$f_4(x_1, x_2, x_3, x_4) = x_1 + x_2$

▶ **4.** Wenn nach einem Programm noch ein weiteres While-Programm ⟨R⟩ kommt, wird die Rückgabe an die Funktion $R_k(x_1,\ldots, x_n)$ deligiert, wobei $R_k(x_1,\ldots, x_n)$ die Reterna-Funktion sein soll, die ⟨R⟩ implementiert.

Beispiel:

```
++x2
--x3
x4 ← x1 + x2
⟨R⟩
```

$f_k(x_1, x_2, x_3, x_4) = R_k(x_1, x_2+1, x_3-1, x_1+x_2)$

▸ 5. *if*-Anweisungen

```
if ⟨Bedingung⟩
   ⟨P⟩
end
⟨R⟩
```

Seien $P_k(x_1, \ldots, x_n)$ und $R_k(x_1, \ldots, x_n)$ Reterna-Funktionen für die While-Programmteile ⟨P⟩ und ⟨R⟩.

Das ganze Programm wird dann berechnet durch:

$f_k(x_1, \ldots, x_n) = \langle Bedingung \rangle \; ? \; R_k(P_1(x_1, \ldots, x_n), \ldots, P_n(x_1, \ldots, x_n)) : R_k(x_1, \ldots, x_n)$

▸ 6. *while*-Anweisungen

```
while ⟨Bedingung⟩
   ⟨P⟩
end
⟨R⟩
```

Das ganze Programm wird berechnet durch:

$f_k(x_1, \ldots, x_n) = \langle Bedingung \rangle \; ? \; f_k(P_1(x_1, \ldots, x_n), \ldots, P_n(x_1, \ldots, x_n)) : R_k(x_1, \ldots, x_n)$

Beispiel:

```
i ← 0
b ← true
while b and (i < len(in))
   if in[i] = "A"
      b ← false
   end
   ++i
end
```

$f(in) = f_3(in, 0, true)$

$f_k(in, i, b) = (b \; and \; (i < len(in))) \; ? \; f_k(g_1(in, i, b), g_2(in, i, b), g_3(in, i, b)) : b$

```
if in[i] = "A"
   b ← false
end
++i
```

$g_1(in, i, b) = in$

$g_2(in, i, b) = i+1$

$g_3(in, i, b) = (in[i] = "A") \; ? \; false : b$

eingesetzt:

$f_k(in, i, b) = (b \; and \; (i < len(in))) \; ? \; f_k(in, i+1, (in[i] = "A" \; ? \; false : b)) : b$

EOL

End Of Lecture

Klausur

26.03.2025 um 13:00 Uhr in der Hessenhalle 1
(Änderungen vorbehalten)

- Seien Sie spätestens um 12:45 Uhr da!
- Bringen Sie mit:
  1. Studentenausweis
  2. Schreibzeug (Stifte)
  3. evtl. unbeschriftetes (!!!!) Notizpapier
  4. evtl. Verpflegung
  5. sonst nix!
- Wenn Sie Hessenhalle betreten:
  1. Sitzplatz suchen (Liste finden Sie im Moodle)
  2. NICHT die Klausur anfassen! (Wir fangen gemeinsam an)
- Dauer: 90 Minuten

▸ Klausuraufbau

(Änderungen vorbehalten)

7 Aufgaben, jeweils zwischen 14 und 20 Punkte, insgesamt 110 Punkte, Bestnote ab ≤ 100 Punkte.

Inhalt: Stoff aus Videos + (einiger) Stoff aus Vorlesungen (welcher, siehe unten)

In der Klausur müssen Sie:

- 3 endliche Automaten zeichnen
- mindestens 3 reguläre Ausdrücke hinschreiben
- 2 kontextfreie Grammatiken angeben
- 2 Loop-/While-Programme schreiben
- 1 Reternafunktion schreiben und 1 verstehen
- mindestens eine Aufgabe mit etwas aus der Vorlesung, das nicht im Video besprochen wurde, bearbeiten
- ein Berechnungsmodell meistern, das Sie noch nie gesehen haben.

Syntaxübersicht wird mitgegeben (siehe auch Moodle)

▸ Klausurvorbereitung

- Videos + Mitschriften
- Vorlesungsmitschriften
- Aufgabensammlung (35 Aufgaben mit Lösungen)
- Aufgaben aus Übungsgruppen/Hausübungen
- Altklausuren

▸ Klausurrelevante Inhalte aus der Vorlesung…

… die nicht im Video besprochen worden sind

- Pumping-Lemma für kontextfreie Grammatiken
- Paare und Listen in Loop/While

- Die Goto-Sprache
- Turingmaschinen
- Allgemeine Grammatiken

Eines dieser Dinge wird in der Klausur vorkommen! 😜